

AI Assisted Coding

Name : G. Usha Sree

H.no : 2303A52431

Batch : 34

Course Code : 23CS002PC304

Assignment Number : 17.2

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts

Task Description – 1(Student Academic Data Cleaning)

- Task: Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

- Handle missing values in marks, attendance, and department.
- Remove duplicate student records based on student ID.
- Standardize text fields such as department names.
- Encode categorical variables like gender and department.

Sample Code:

```
import pandas as pd  
data = pd.read_csv("students.csv")  
print(data.head())
```

Expected Output:

- A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.


```
import pandas as pd
```

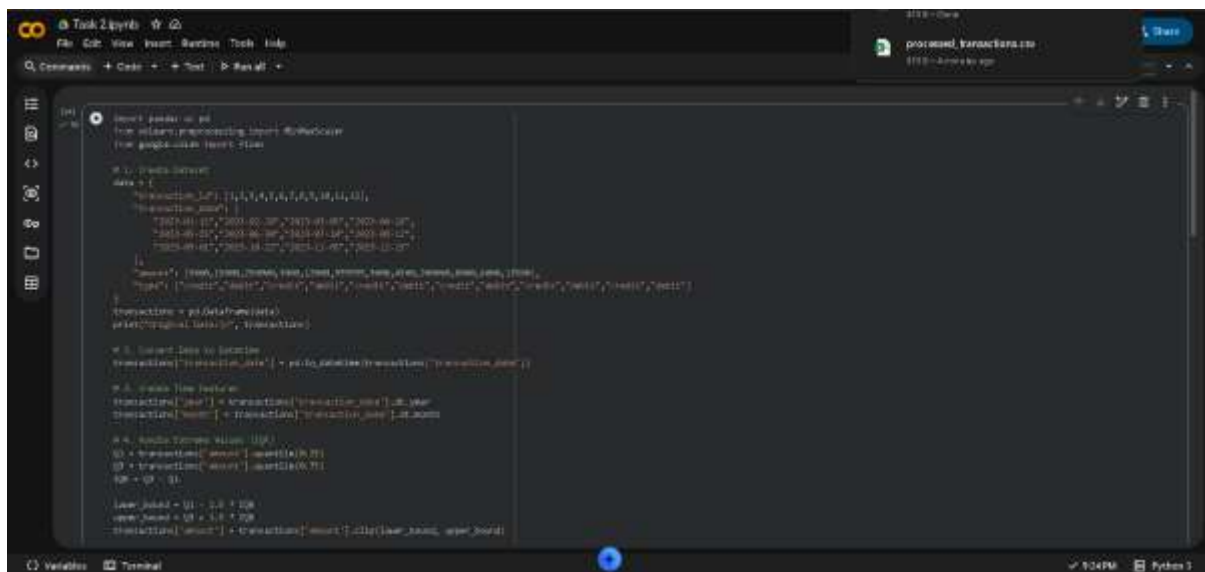
```
from sklearn.preprocessing import MinMaxScaler
```

```
transactions = pd.read_csv("transactions.csv")
```

```
print(transactions.info())
```

Expected Output:

- A transformed dataset with normalized values and newly generated time based features.



```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from datetime import datetime

# 1. Create Dataset
data = [
    "transaction_id", [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "transaction_amount", [
        "2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04",
        "2023-01-05", "2023-01-06", "2023-01-07", "2023-01-08",
        "2023-01-09", "2023-01-10", "2023-01-11", "2023-01-12"
    ],
    "amount", [1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000, 5500, 6000, 6500],
    "type", ["credit", "debit", "credit", "debit", "credit", "debit", "credit", "debit", "credit", "debit", "credit", "debit"]
]

transactions = pd.DataFrame(data)
print(transactions.info())

# 2. Convert Date to datetime
transactions["transaction_date"] = pd.to_datetime(transactions["transaction_date"])

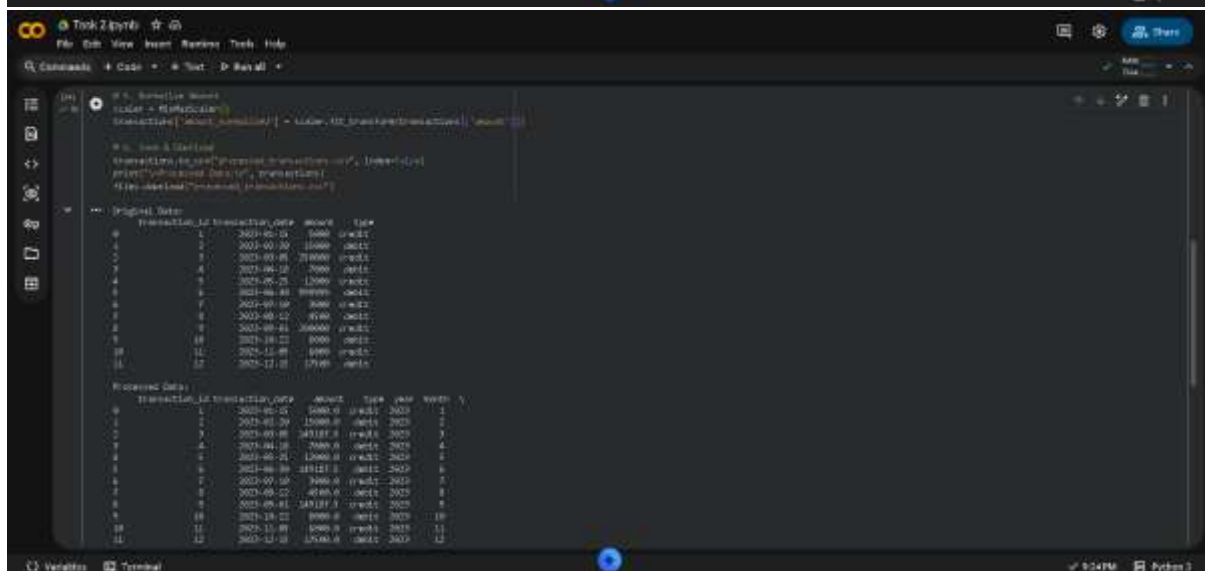
# 3. Create Time Features
transactions["year"] = transactions["transaction_date"].dt.year
transactions["month"] = transactions["transaction_date"].dt.month

# 4. Add Timezone Alias (UTC)
tz = transactions["year"].astype(str)
tz = transactions["month"].astype(str)
tz = tz + "-"

# 5. Add Timezone Alias (UTC)
tz = tz + "UTC"

# 6. Add Timezone Alias (UTC)
tz = tz + "UTC"

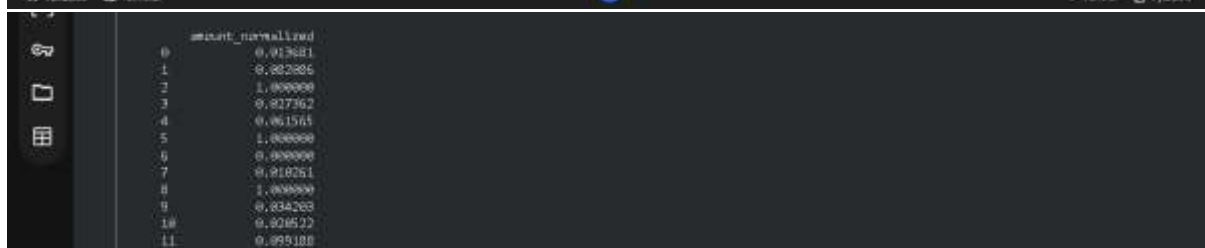
# 7. Add Timezone Alias (UTC)
tz = tz + "UTC"
```



```
# 8. Apply MinMaxScaler
scaler = MinMaxScaler()
transactions["amount_normalized"] = scaler.fit_transform(transactions["amount"])

# 9. View & Display
transactions.to_csv("processed_transactions.csv", index=False)
print("Processed Data:", transactions)
file_path = "processed_transactions.csv"
```

transaction_id	transaction_date	amount	type
1	2023-01-01	1000	credit
2	2023-01-02	1500	debit
3	2023-01-03	2000	credit
4	2023-01-04	2500	debit
5	2023-01-05	3000	credit
6	2023-01-06	3500	debit
7	2023-01-07	4000	credit
8	2023-01-08	4500	debit
9	2023-01-09	5000	credit
10	2023-01-10	5500	debit
11	2023-01-11	6000	credit
12	2023-01-12	6500	debit



```
amount_normalized
```

amount_normalized	
0	0.013681
1	0.020521
2	0.027362
3	0.034203
4	0.041043
5	0.047884
6	0.054725
7	0.061565
8	0.068406
9	0.075247
10	0.082087
11	0.088928

Task 3 – (Environmental Sensor Data Preparation)

- Task: Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- Handle missing values in temperature, humidity, and air_quality_index.
- Scale numerical features using standard scaling.
- Encode categorical fields such as sensor_status.
- Split the dataset into training and testing subsets.

Sample Code:

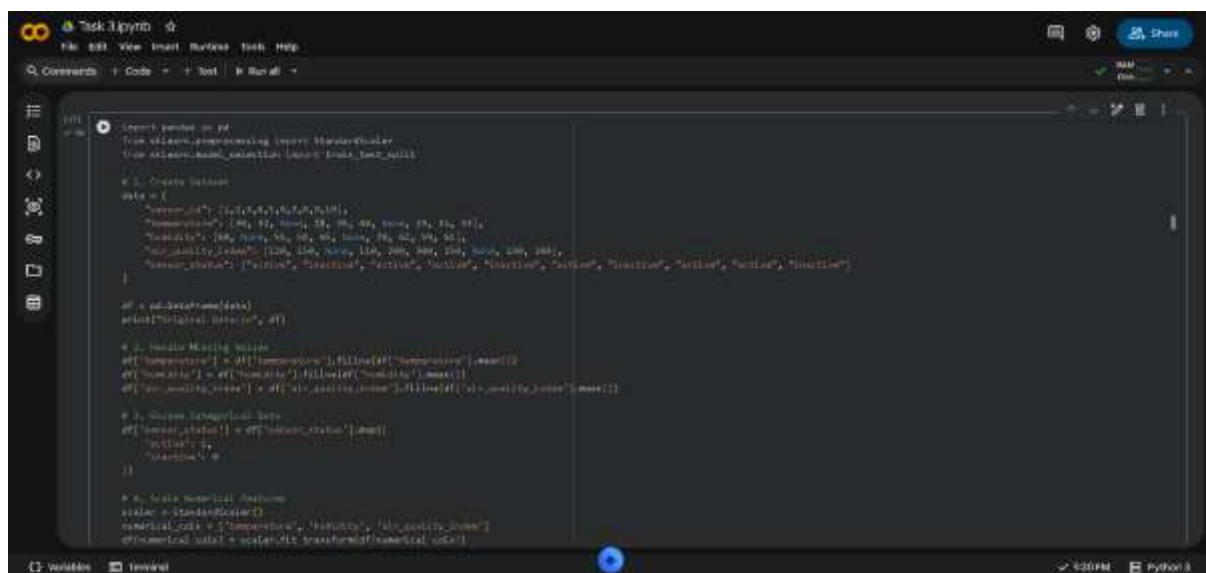
```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("sensor_data.csv")
```

Expected Output:

- A fully preprocessed dataset ready for predictive modeling tasks.



```
Task 3.ipynb
File Edit View Insert Cell Help
In [ ]:
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. Create Dataset
data = {
    "sensor_id": [101, 102, 103, 104, 105],
    "temperature": [25, 28, None, 22, 26, 29, 27, 24],
    "humidity": [60, 65, 68, 62, 70, 72, 69, 66],
    "air_quality_index": [120, 130, None, 110, 140, 135, 125, 145],
    "sensor_status": ["Active", "Inactive", "Active", "Active", "Inactive", "Active", "Inactive", "Active"]
}

df = pd.DataFrame(data)
print("Initial Dataset:", df)

# 2. Handle Missing Values
df["temperature"] = df["temperature"].fillna(df["temperature"].mean())
df["humidity"] = df["humidity"].fillna(df["humidity"].mean())
df["air_quality_index"] = df["air_quality_index"].fillna(df["air_quality_index"].mean())

# 3. Encode Categorical Data
df["sensor_status"] = df["sensor_status"].map(
    {"Active": 1,
     "Inactive": 0}
)

# 4. Scale Numerical Features
scaler = StandardScaler()
numerical_cols = ["temperature", "humidity", "air_quality_index"]
df[numerical_cols] = scaler.fit_transform(numerical_cols)
```

```

# 3. Split Dataset
X = df.drop(columns='sensor_id') # Features
y = df['sensor_status'] # Target

# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)

# 5. Feature Scaling
print(f"Normalized Data: {X_train}")
print(f"Training Set Shape: {X_train.shape}")
print(f"Testing Set Shape: {X_test.shape}")

# Original Data:
sensor_id temperature humidity air_quality_index sensor_status
0 1 85.0 65.0 120.0 inactive
1 2 72.0 75.0 150.0 inactive
2 3 90.0 55.0 80.0 active
3 4 10.0 10.0 110.0 active
4 5 95.0 85.0 200.0 inactive
5 6 40.0 5.0 900.0 inactive
6 7 90.0 70.0 250.0 inactive
7 8 10.0 65.0 90.0 active
8 9 81.0 66.0 100.0 active
9 10 77.0 81.0 100.0 inactive

# Normalized Data:
sensor_id temperature humidity air_quality_index sensor_status
0 1 -0.666770 -0.316222 -0.557123 1
1 2 -0.657709 0.000000 -0.430108 0
2 3 0.000000 -1.027351 0.660000 1
3 4 -1.132046 -0.584218 -1.132046 1
4 5 0.154296 -0.376417 0.550208 0
5 6 2.400271 0.000000 2.250071 1
6 7 0.000000 1.177390 1.307082 0
7 8 -1.016214 0.195235 0.660000 1
8 9 -0.186944 -0.548494 -0.715449 1
9 10 0.223120 -0.305994 -0.865272 0

Training Set Shape: (8, 4)
Testing Set Shape: (2, 4)

```

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

• Scenario:

A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- Standardize road and location names.
- Fill missing traffic density values using appropriate statistical methods.
- Remove duplicate sensor readings.
- Normalize speed and vehicle count metrics.
- Generate a brief summary comparing dataset quality before and after cleaning

Sample Code:

```
import pandas as pd

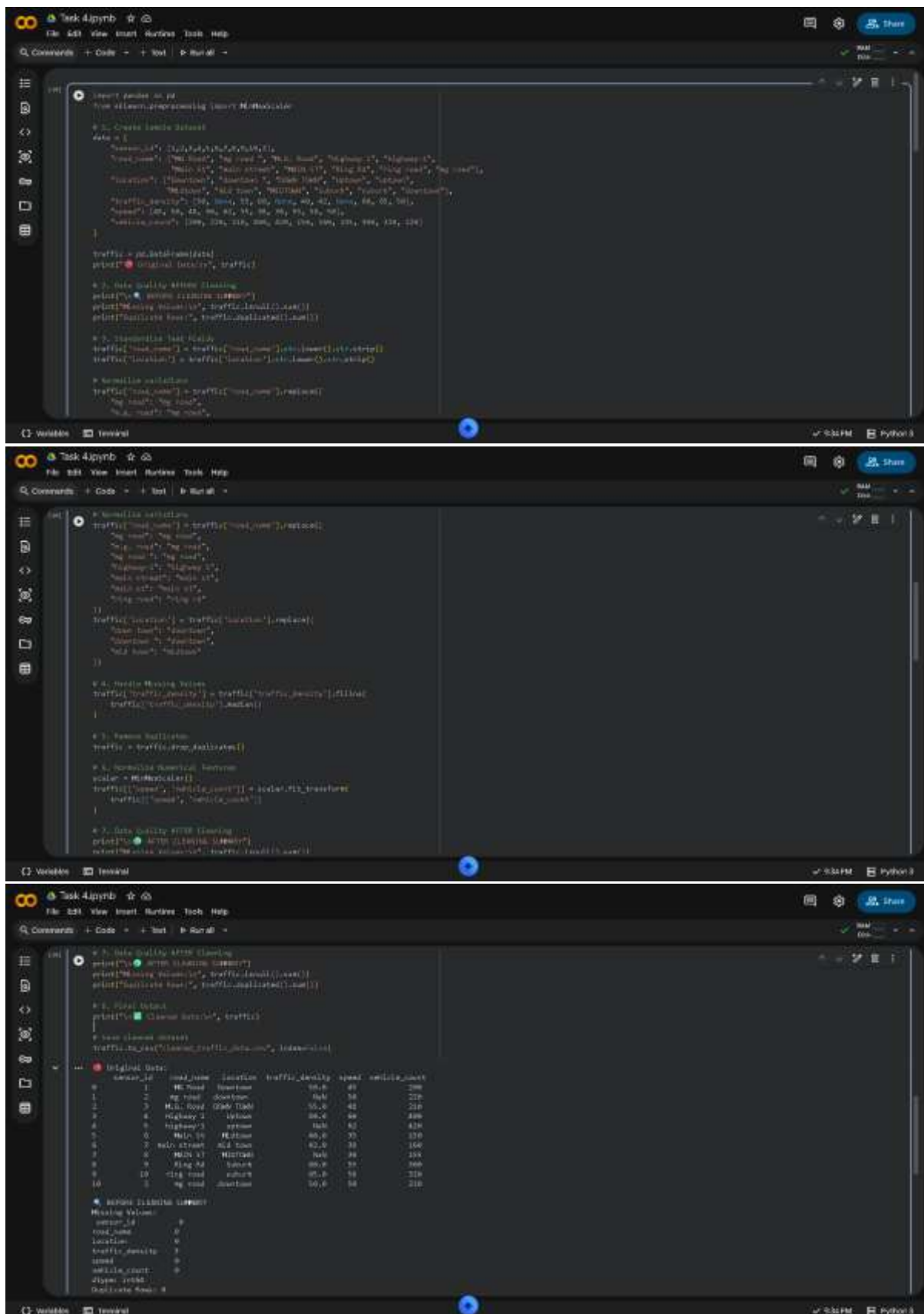
traffic = pd.read_csv("traffic_data.csv")

print(traffic.describe())
```

Expected Output:

- A cleaned and normalized traffic dataset with an accompanying

data-quality improvement summary.



The image displays three sequential screenshots of a Jupyter Notebook interface, showing the progression of data quality improvement steps. The notebook is titled "Task 4.ipynb" and is running on a Python 3 environment.

First Screenshot: The notebook shows the initial data loading and preprocessing steps. The code includes:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 1. Create sample dataset
data = {
    'sensor_id': [1,2,3,4,5,6,7,8,9,10],
    'road_name': ['M1 Road', 'M20 Road', 'M10 Road', 'Ring Road', 'M10 Road', 'M20 Road', 'M10 Road', 'M20 Road', 'M10 Road', 'M20 Road'],
    'location': ['London', 'London', 'London', 'London', 'London', 'London', 'London', 'London', 'London', 'London'],
    'traffic_density': [10, 20, 30, 40, 50, 60, 70, 80, 90, 100],
    'speed': [10, 20, 30, 40, 50, 60, 70, 80, 90, 100],
    'vehicle_count': [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
}

traffic = pd.DataFrame(data)

print('Original Data:', traffic)
```

Second Screenshot: The notebook shows the data cleaning and normalization steps. The code includes:

```
# 2. Data Quality Error Fixing
print('Before cleaning dataset')
print('Missing Values:', traffic.isnull().sum())
print('Duplicate Rows:', traffic.duplicated().sum())

# 3. Normalise the data
traffic['road_name'] = traffic['road_name'].str.lower()
traffic['location'] = traffic['location'].str.lower()

# 4. Normalise values
traffic['road_name'] = traffic['road_name'].replace(
    {'M1 Road': 'M1', 'M20 Road': 'M20', 'M10 Road': 'M10', 'Ring Road': 'Ring', 'M20 Road': 'M20', 'M10 Road': 'M10', 'M20 Road': 'M20', 'M10 Road': 'M10', 'M20 Road': 'M20'})

# 5. Handle Missing Values
traffic['traffic_density'] = traffic['traffic_density'].fillna(
    traffic['traffic_density'].mean())

# 6. Remove Duplicates
traffic = traffic.drop_duplicates()

# 7. Normalise numerical features
scaler = MinMaxScaler()
traffic[['sensor_id', 'road_name']] = scaler.fit_transform(
    traffic[['sensor_id', 'road_name']])

# 8. Data Quality Error Fixing
print('After cleaning dataset')
print('Missing Values:', traffic.isnull().sum())
print('Duplicate Rows:', traffic.duplicated().sum())
```

Third Screenshot: The notebook shows the final data quality improvement steps. The code includes:

```
# 9. Final Output
print('Cleaned Data:', traffic)
```

The final output of the notebook shows the cleaned and normalized data, including the original data and the cleaned data. The output is displayed as a table with columns: sensor_id, road_name, location, traffic_density, speed, and vehicle_count.

sensor_id	road_name	location	traffic_density	speed	vehicle_count
0	1	M1 Road	London	10.0	100
1	2	M20 Road	London	20.0	200
2	3	M10 Road	London	30.0	300
3	4	Ring Road	London	40.0	400
4	5	M1 Road	London	50.0	500
5	6	M20 Road	London	60.0	600
6	7	M10 Road	London	70.0	700
7	8	Ring Road	London	80.0	800
8	9	M1 Road	London	90.0	900
9	10	M20 Road	London	100.0	1000

The screenshot shows a Jupyter Notebook window titled 'Task 4.ipynb'. The notebook contains a single cell with a DataFrame named 'cleaned_data'. The DataFrame has 11 columns: 'sensor_id', 'road_name', 'location', 'traffic_density', 'speed', and 'vehicle_count'. The data is as follows:

	sensor_id	road_name	location	traffic_density	speed	vehicle_count
0	1	mg road	eastern	88.8	0.976678	0.225180
1	2	mg road	eastern	82.5	0.885538	0.278270
2	3	mg road	eastern	86.0	0.401446	0.112222
3	4	highway 1	central	88.0	0.825638	0.830000
4	5	highway 1	central	81.5	1.888888	1.000000
5	6	highway 1	central	88.8	0.888888	0.888888
6	7	highway 1	central	82.8	0.111111	0.097007
7	8	highway 1	central	81.5	0.837007	0.822149
8	9	highway 1	central	88.8	0.808081	0.830000
9	10	highway 1	central	88.8	0.808081	0.828000
10	1	mg road	eastern	86.0	0.855556	0.278270

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

• Task:

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- Remove duplicate posts and null text entries.
- Clean text by removing URLs, special characters, and extra spaces.
- Convert text to lowercase and perform basic tokenization.
- Remove stopwords and perform stemming or lemmatization. Generate basic text features such as word count and sentiment score.

Sample Code:

```
import pandas as pd
import re

posts = pd.read_csv("social_media_posts.csv")
print(posts.head())
```

Expected Output:

A cleaned text dataset with extracted features suitable for sentiment analysis models.

